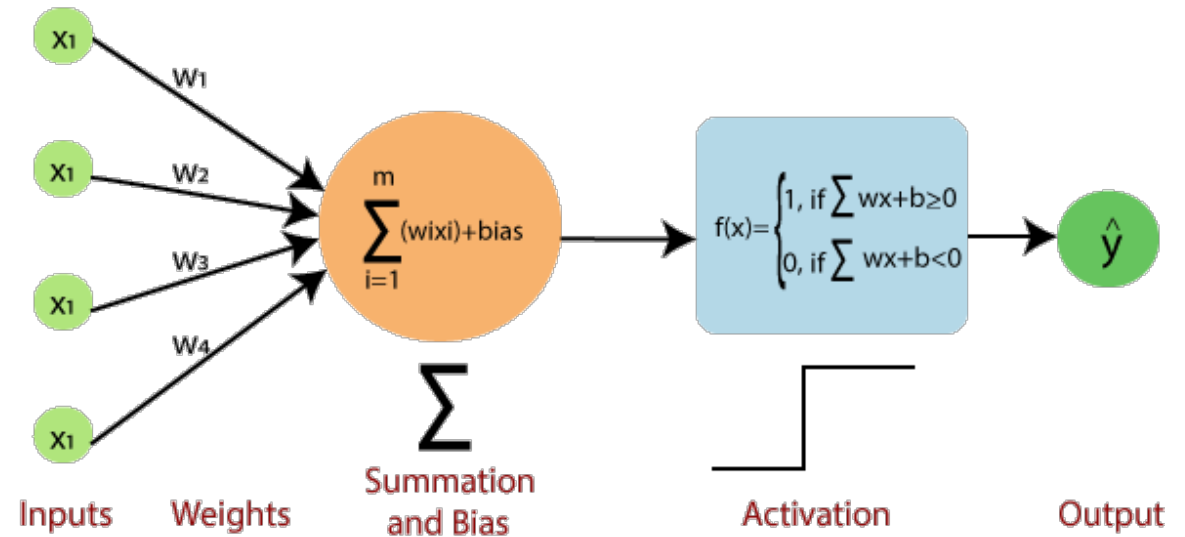
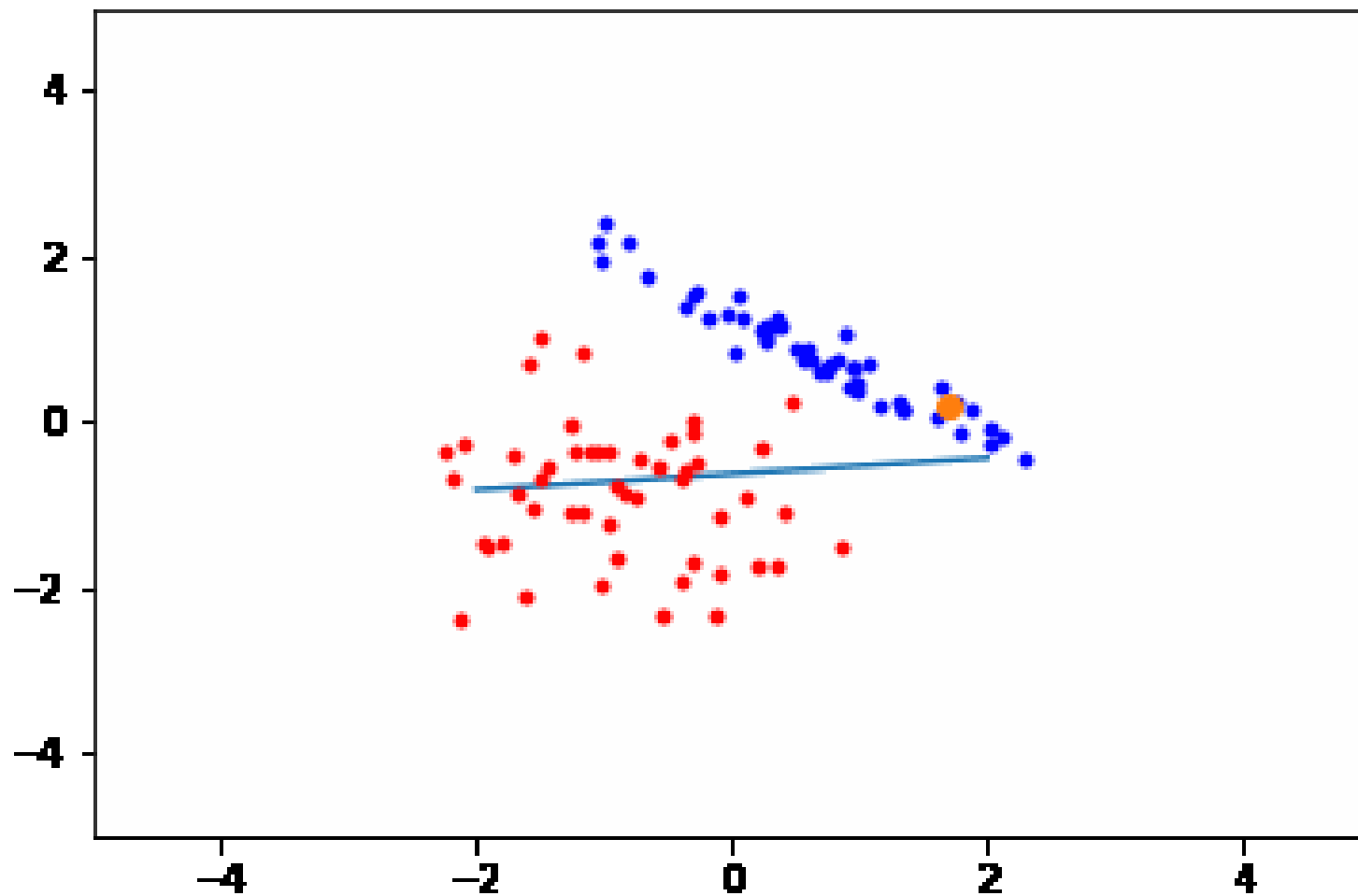


The Perceptron

- Input: feature vector $x \in \mathbb{R}^m$
- Label: $y \in \{0, 1\}$
- Model:
$$\hat{y} = \sigma(w^T x + b)$$
- **Update rule:** $\Delta w = \eta(y - \hat{y})x$
- **Intuition:**
 - w is an hyperplane
 - We include or exclude the input based on difference from the label

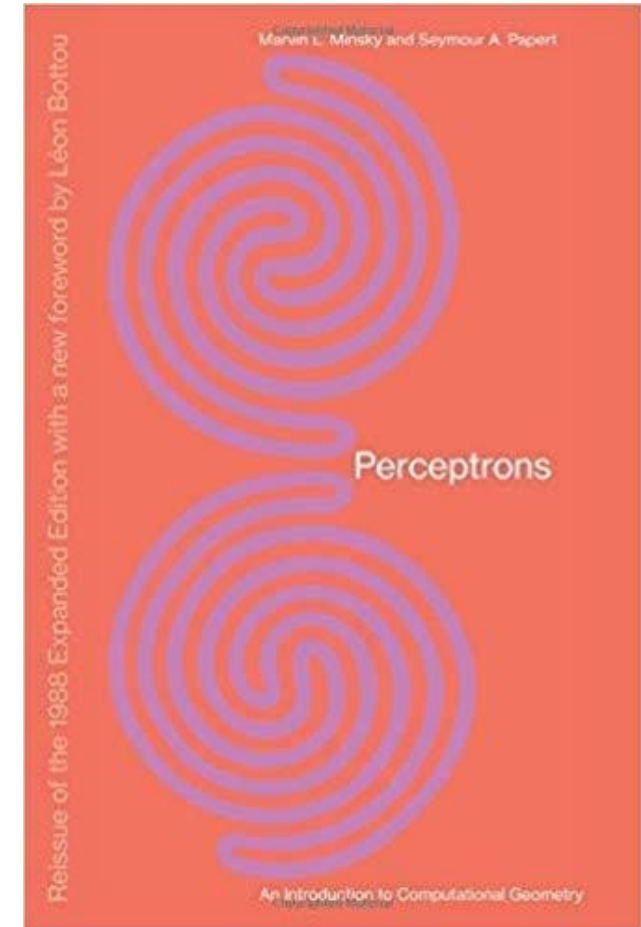
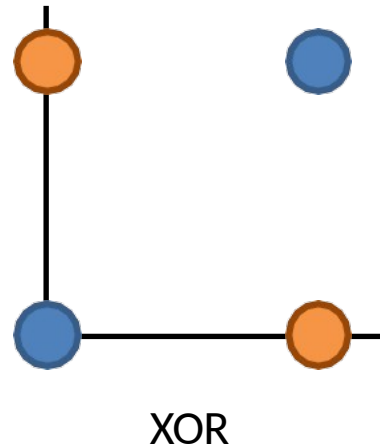


Perceptron learning: the line moves toward misclassified points



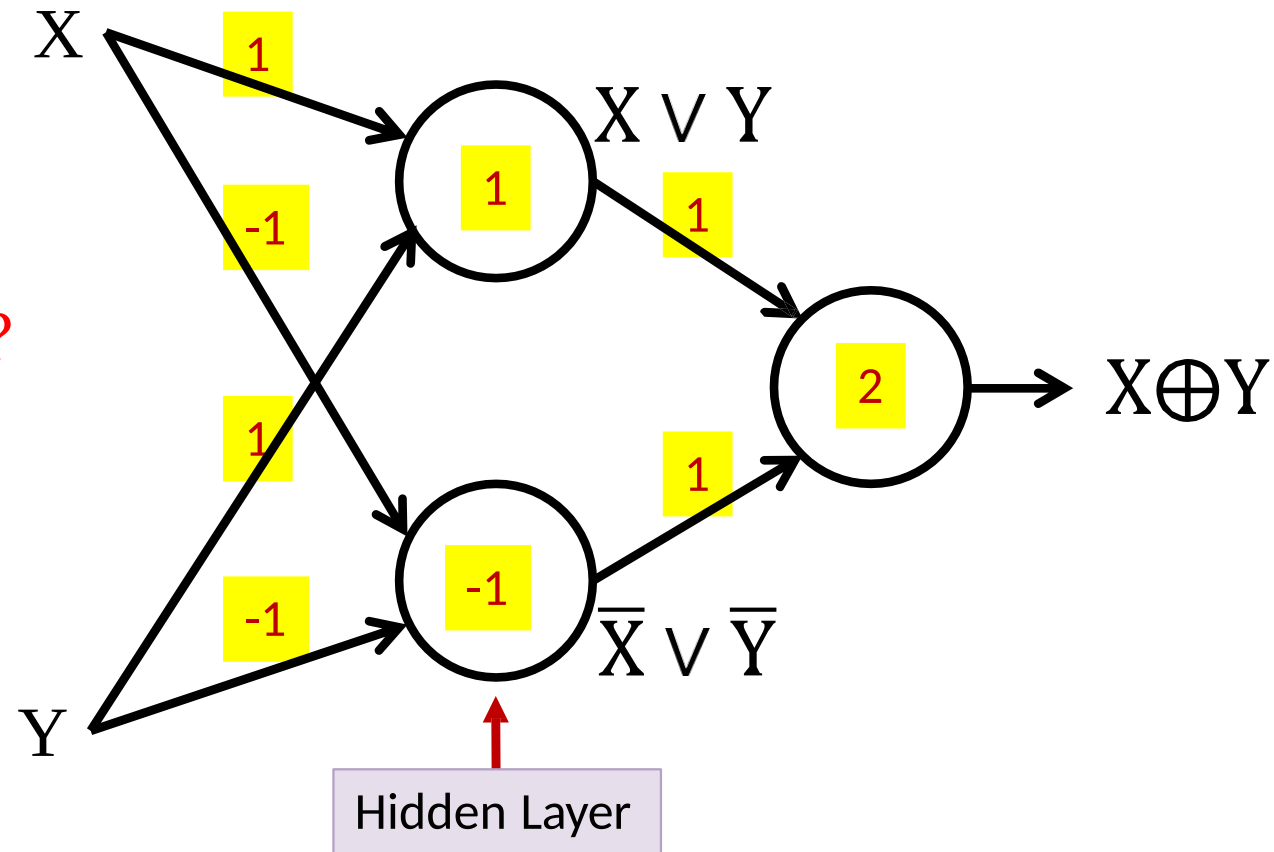
Minsky and Papert, 1969 (AI Winter)

- Showed that Perceptron could not learn the XOR function

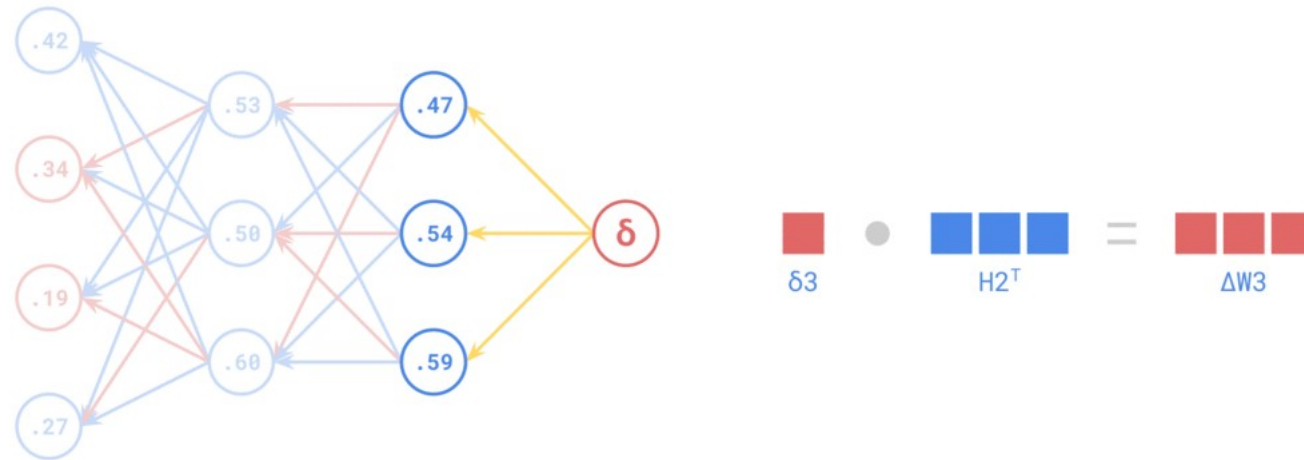


XOR and Multi-layer Perceptron (MLP)

But how to train?

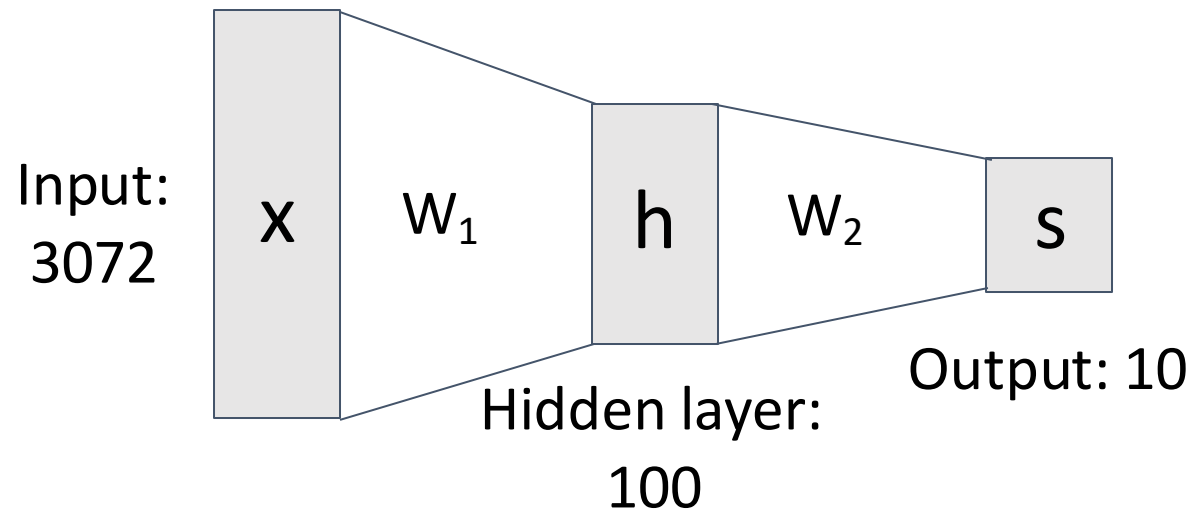


Backpropagation for MLP



Multi-Layer Perceptron (MLP)

- **(Before)** Linear score function: $f = Wx$
- **(Now)** 2-layer Neural Network: $f = W_2 \max(0, W_1 x)$



$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

Activation Functions

- This is called the **activation function** of the neural network

$$f = W_2 \max(0, W_1 x)$$

- **Q:** What happens if we build a neural network with no activation function?

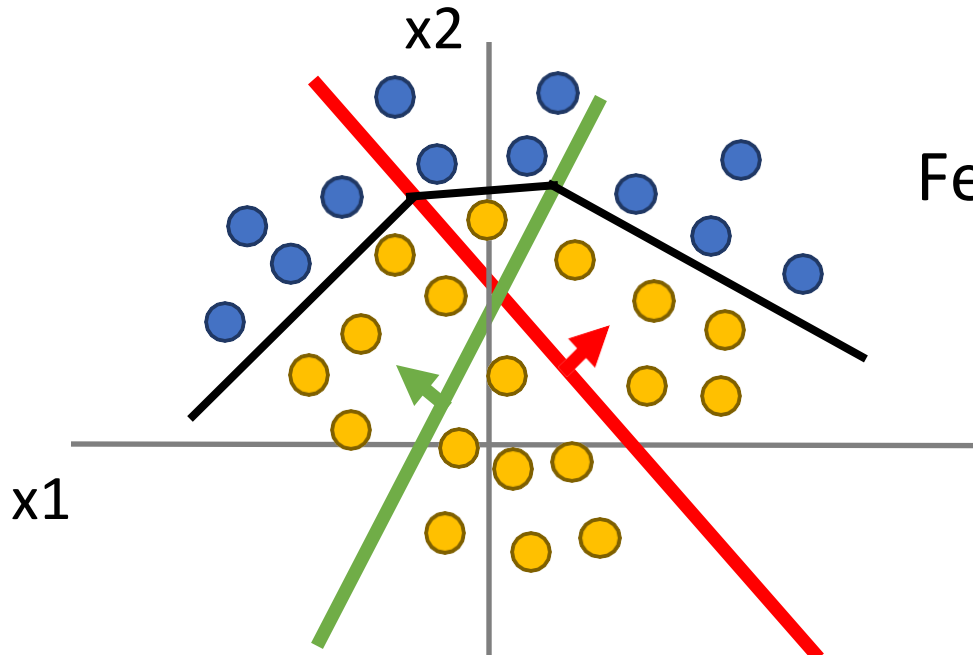
$$s = W_2 W_1 x$$

$$W_3 = W_2 W_1 \in \mathbb{R}^{C \times H} \quad s = W_3 x$$

- **A:** We end up with a linear classifier!

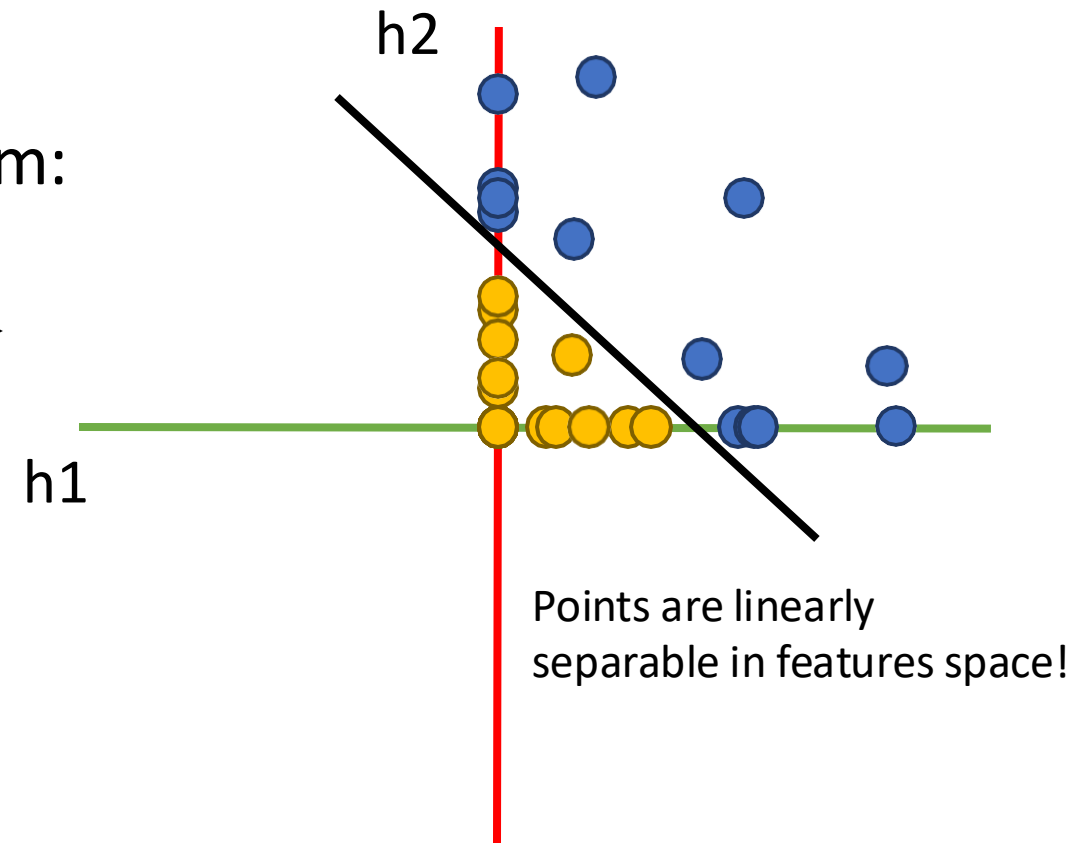
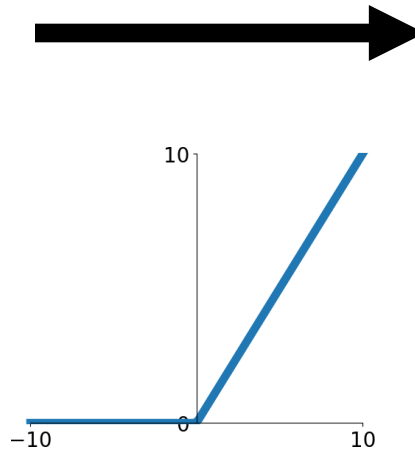
Nonlinear boundary

Points not linearly separable in original space

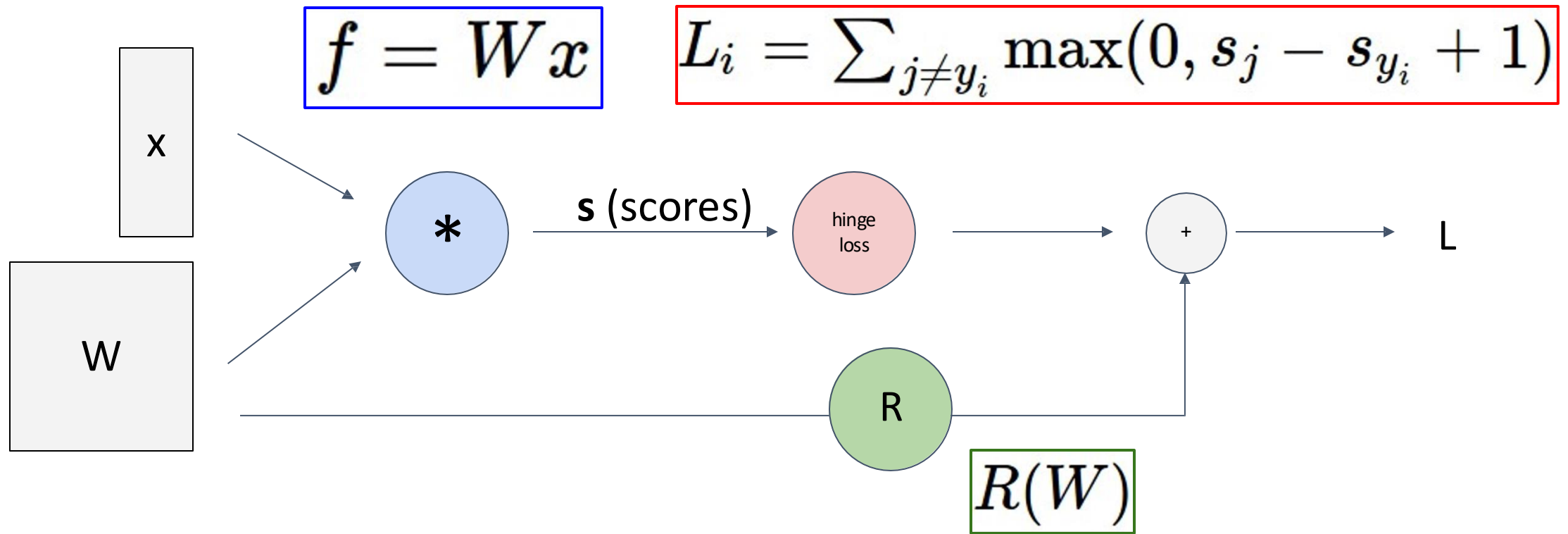


Linear classifier in feature space gives nonlinear classifier in original space

Feature transform:
 $h = \text{ReLU}(Wx)$

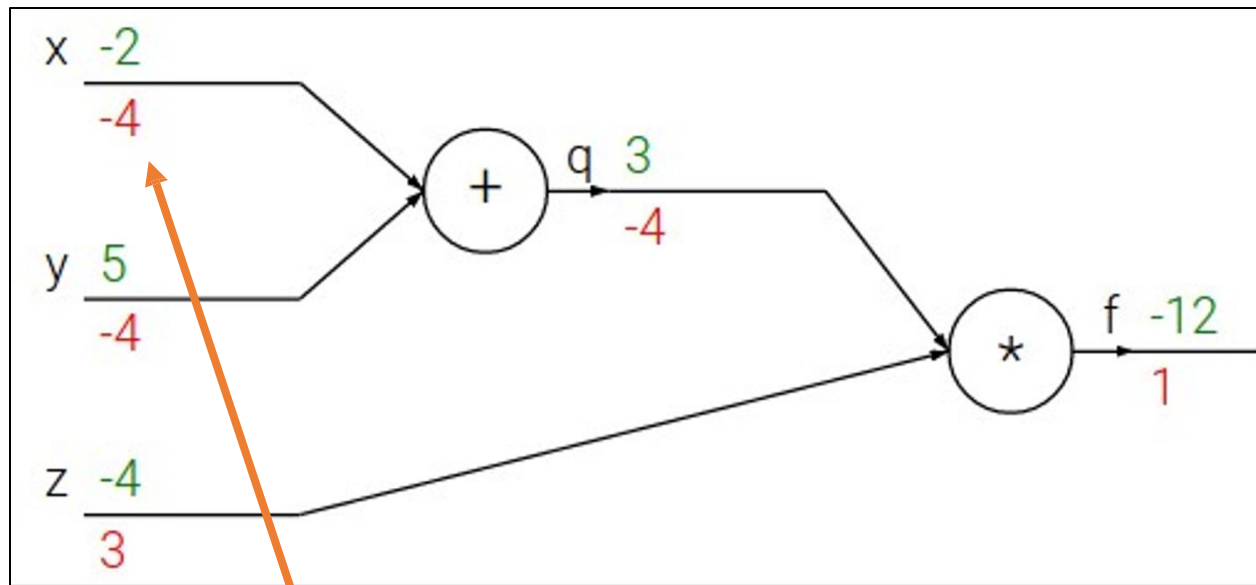


Backpropagation Graphs



Forward pass computes outputs

Backward pass computes gradients



Chain Rule

$$q = x + y$$

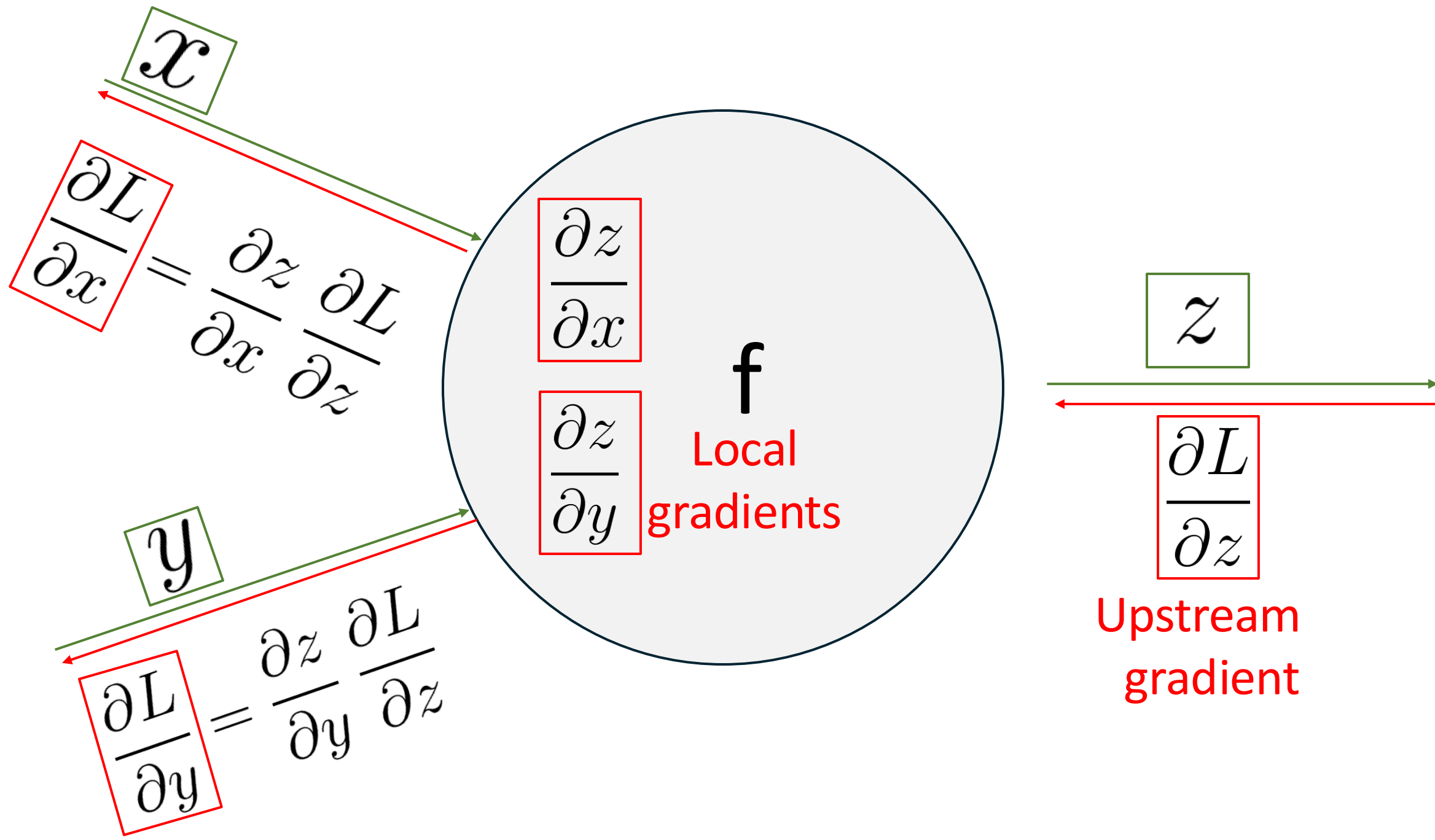
$$\frac{\partial f}{\partial x} = \frac{\partial q}{\partial x} \frac{\partial f}{\partial q}$$

$$\frac{\partial q}{\partial x} = 1$$

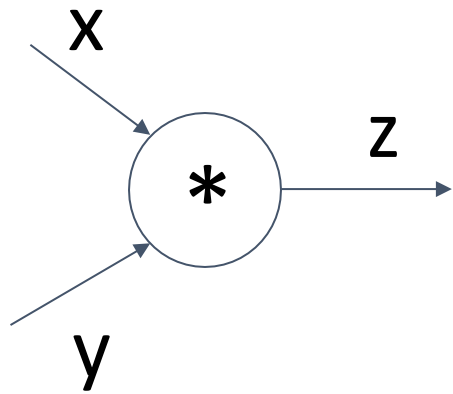
**Downstream
Gradient**

**Local
Gradient**

**Upstream
Gradient**



Example: PyTorch Autograd Functions



(x,y,z are scalars)

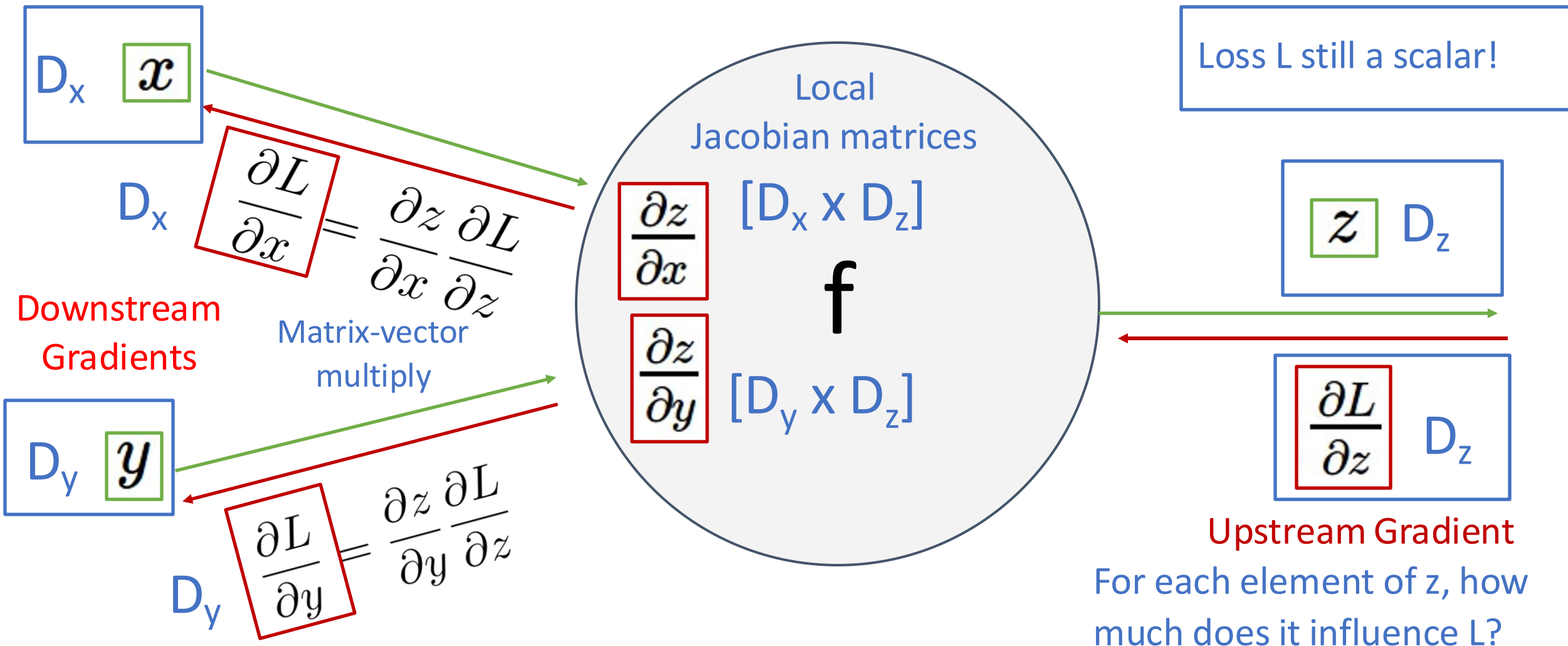
```
class Multiply(torch.autograd.Function):  
    @staticmethod  
    def forward(ctx, x, y):  
        ctx.save_for_backward(x, y)  
        z = x * y  
        return z  
    @staticmethod  
    def backward(ctx, grad_z):  
        x, y = ctx.saved_tensors  
        grad_x = y * grad_z    # dz/dx * dL/dz  
        grad_y = x * grad_z    # dz/dy * dL/dz  
        return grad_x, grad_y
```

Need to stash some
values for use in
backward

Upstream
gradient

Multiply upstream
and local gradients

Backprop with Vectors



Backprop with Vectors

Jacobian is **sparse**: off-diagonal entries all zero! Never **explicitly** form Jacobian; instead use **implicit** multiplication

4D input x:

$\begin{bmatrix} 1 \\ -2 \\ 3 \\ -1 \end{bmatrix}$

$$f(x) = \max(0, x) \\ (\textit{elementwise})$$

4D output y:

$\begin{bmatrix} 1 \\ 0 \\ 3 \\ 0 \end{bmatrix}$

4D dL/dx:

$\begin{bmatrix} 4 \\ 0 \\ 5 \\ 0 \end{bmatrix}$

$[dy/dx] [dL/dy]$

$$\left(\frac{\partial L}{\partial x} \right)_i = \begin{cases} \left(\frac{\partial L}{\partial y} \right)_i & \text{if } x_i > 0 \\ 0 & \text{otherwise} \end{cases}$$

4D dL/dy:

$\begin{bmatrix} 4 \\ -1 \\ 5 \\ 9 \end{bmatrix}$

Upstream
gradient